

Adding complexity

Namkha advanced tutorial

Centre for Research into Ecological & Environmental Modelling
University of St Andrews

Session 3 objectives

Main goal: Show where the simple workflow has to change when the analysis gets more realistic.

1. Longer survey periods and closure
2. Shortened-survey and multisession alternatives
3. Elevation constraints and reporting masks
4. Gorge-avoiding movement distances
5. Sex-specific model components

Assumption: You have already seen the simple workflow from Session 2.

From simple to advanced

The basic workflow stays the same:

Each extension affects a specific part of the workflow.

01

camera file
detection
file

⇒

capthist

02

survey area
buffer
covariates

⇒

mask

03

(crop for
multisession
model)

⇒

mask (ms)

04

D model
 λ_0/g_0
model

⇒

seccr.fit

05

AICc
abundance
density
maps

Extension 1: Long surveys and closure

Extension 1: Long surveys and closure

Problem: the full survey is long enough that the closed-population assumption may be questionable.

Implementation:

- keep the full survey
- create a shortened survey window
- split the full survey into temporal sessions
- compare how results change

Scripts involved:

- 01_make_capthist.R
- 03_make_ms_masks.R
- 04a, 04b, 04c
- 05a, 05b, 05c

Full survey: occasion-specific usage

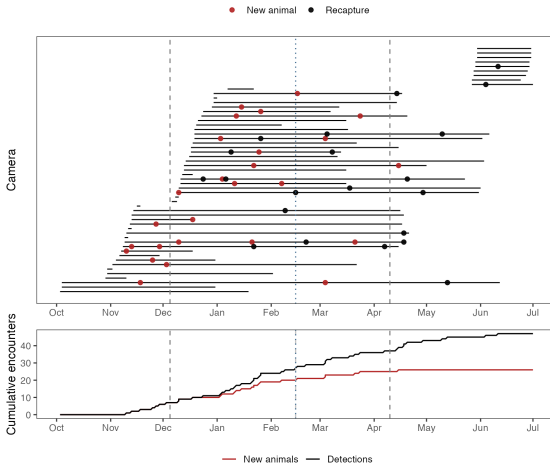
secr implements closure checks. These need multiple occasions (no other need for these i.e. for model fitting)

```
full_occ_length <- 14
full_occ_start <- seq.Date(from = survey_start, to = survey_end,
                           by = paste(full_occ_length, "days"))
full_occ_bins <- interval(start = full_occ_start,
                          end = full_occ_end)
```

For each camera and occasion, need to recalculate effort (days active)

```
1 usage_full <- matrix(0, nrow = nrow(cameras), ncol = length(full_occ_bins))
2 for (i in seq_len(nrow(cameras))) {
3   usage_full[i, ] <- round(as.numeric(
4     int_overlaps_numeric(cameras$camera_on[i], full_occ_bins)) / (24 * 60 * 60)
5   )
6   usage(full_traps) <- usage_full
```

Full survey: descriptive check



- cameras started at different times
- new animals appear through time
- shortened window is a defensible alternative
- session split is another alternative

Full survey: closure checks

The full survey occasions allow a formal closure check

```
closure.test(full_ch)
```

statistic	p
-1.56677248	0.05858394

Suggests moderate concern about closure. Now check closure for the shortened survey

```
short_occasion_start <- floor(as.numeric(short_survey_start - survey_start) /  
                             full_occ_length) + 1  
short_occasion_end <- floor(as.numeric(short_survey_end - survey_start) /  
                             full_occ_length) + 1  
closure.test(subset(full_ch, occasions = short_occasion_start:short_occasion_end))
```

statistic	p
-0.06098922	0.47568390

Shortened survey: workflow change

Key idea: limit survey length by taking a subset of the full survey

Issue: Which part of the full survey to take?

Keep only detections in the shortened window

```
short_captcs_raw <- captcs_raw |>
  filter(date >= short_survey_start, date <= short_survey_end)
```

Recalculate camera effort for that same window

```
short_traps_df <- cameras |>
  mutate(
    effort_short = pmax(
      0,
      as.numeric(pmin(end_date, short_survey_end) -
                    pmax(start_date, short_survey_start), units = "days")
    )
  )
```

Shortened survey: capthist

Create and check the capture history

```
1 short_capt <- short_capt_raw |>
2   mutate(occasion = 1L) |>
3   dplyr::select(session, animalID, occasion, trapID, sex)
4
5 short_ch <- make.capthist(captures = short_capt, traps = short_traps)
6 verify(short_ch)
7 summary(short_ch, terse = TRUE)
```

Occasions	Detections	Animals	Detectors
1	30	21	55

Multisession survey: workflow change

Key idea: keep the full survey but split it into temporal sessions.

Detections get a session label:

```
ms_captvs <- captvs_raw |>
  mutate(
    session = if_else(date < ms_session_end, 1L, 2L),
    occasion = 1L
  ) |> dplyr::select(session, animalID, occasion, trapID, sex)
```

Camera effort is calculated separately for each session:

```
ms_traps_df <- cameras |>
  mutate(
    effort_s1 = pmax(0, as.numeric(pmin(end_date, ms_session_end - days(1)) -
                                     pmax(start_date, survey_start),
                                     units = "days")),
    effort_s2 = pmax(0, as.numeric(pmin(end_date, survey_end) -
                                     pmax(start_date, ms_session_end),
                                     units = "days"))
  )
```

Multisession survey: capthist

For multisession secr, the traps object is a named list:

```
ms_traps_list <- split(ms_traps_df, f = ms_traps_df$session)
traps_ms <- vector("list", length(ms_traps_list))

for (i in seq_along(ms_traps_list)) {
  traps_ms[[i]] <- read.traps(
    data = ms_traps_list[[i]] |> dplyr::select(trapID, x, y, effort),
    detector = "count",
    trapID = "trapID",
    binary.usage = FALSE
  )
}
names(traps_ms) <- c("1", "2")
```

Then add covariates, etc.

Multisession survey: factor levels

Create and check the multisession capture history:

```
ch_ms <- make.capthist(captures = ms_capt, traps = traps_ms)
verify(ch_ms)
```

Make sure to share factor levels across sessions:

```
1 ch_ms <- shareFactorLevels(ch_ms, stringsAsFactors = FALSE)
2 verify(ch_ms)
3 summary(ch_ms, terse = TRUE)
```

	Occasions	Detections	Animals	Detectors
1	1	26	20	55
2	1	21	16	55

Extension 2: Elevation constraints and reporting masks

Extension 2: Elevation constraints

Problem: not every point in the trap buffer is plausible snow leopard habitat.

Implementation:

- add elevation to the full mask
- restrict mask points to plausible elevation range
- keep separate masks for fitting and reporting
- save scaling information for covariate effect plots

Add elevation to the mask

Elevation is downloaded and saved if not available already:

```
elev_file <- "namkha_advanced/output/cache/namkha_elevation.tif"

if (file.exists(elev_file)) {
  elev <- rast(elev_file)
} else {
  elev_download <- get_elev_raster(locations = full_region_4326,
                                   z = 10, clip = "locations")

  elev <- rast(elev_download)
  writeRaster(elev, elev_file, overwrite = TRUE)
}
```

Then project and attach it to the mask:

```
elev <- project(elev, y = my_crs)
names(elev) <- "elev"
mask <- addCovariates(mask, elev)
```

Restrict masks by elevation

We use 3000–5500 m as the plausible elevation range.

```
inside_elev_range <- covariates(mask)$elev >= elev_min &  
  covariates(mask)$elev <= elev_max  
mask_elev <- subset(mask, subset = inside_elev_range)
```

Then create reporting and fitting masks.

```
mask_survey_area_elev <- subset(mask_elev, subset = inside_survey_area_elev)  
mask_model <- subset(mask_elev, subset = inside_buffer_elev)
```

Fitting mask versus reporting mask

Fitting mask: `mask_model`

- elevation-constrained
- within the trap buffer
- can include activity centres outside Namkha

Reporting mask: `mask_survey_area_elev`

- elevation-constrained
- inside the Namkha survey boundary
- used for abundance, density, maps, and covariate summaries

Same conceptual split as Session 2, but now with an extra habitat constraint.

Extension 3: Barriers and custom movement distances

Extension 3: Gorge-avoiding movement

Problem: Euclidean distance says animals can move straight through the gorge.

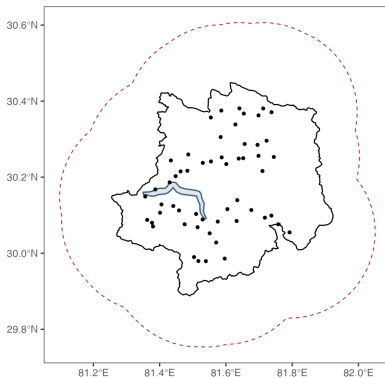
Implementation:

- read the gorge geometry
- make a fine mask for path calculations
- calculate shortest-path distances with `nedist()`
- pass those distances to `secr.fit()` using details

Read the gorge

The gorge is a polygon that must be in the same CRS as traps and masks.

```
gorge <- st_read("../namkha_gorge.kml") |>  
  st_transform(my_crs) |>  
  st_geometry()
```



Fine mask for movement distances

Gorge is narrower than existing mask spacing, so shortest-path calculations will be inaccurate. Use a finer mask to calculate distances around the gorge.

```
mask_fine <- make.mask(  
  traps = full_traps,  
  buffer = mask_buffer,  
  type = "trapbuffer",  
  spacing = 200,  
  poly = gorge,  
  poly.habitat = FALSE,  
  keep.poly = FALSE  
)
```

Note: Still use the normal model mask for model fitting.

Non-Euclidean distance matrix

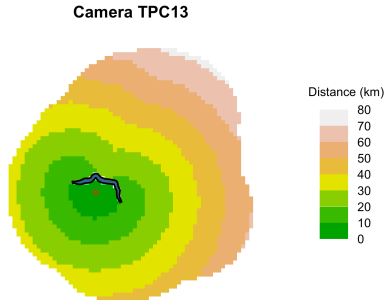
`nedist()` calculates distances from traps to mask points, using the fine mask as the movement network.

```
1 userd <- nedist(  
2   full_traps,  
3   mask,  
4   mask_fine,  
5   directions = 16  
6 )
```

Each cropped mask needs its own `userdist` (this is just a subsetting of rows and columns of the full `userdist`)

```
userd_elev <- userd[, inside_elev_range, drop = FALSE]  
userd_model <- userd_elev[, inside_buffer_elev, drop = FALSE]  
userd_survey_area_elev <- userd_elev[, inside_survey_area_elev, drop = FALSE]
```

Non-Euclidean distance check



- distance from camera 'TPC13' to each mask point
- points across the gorge are farther away than straight-line distance suggests

Use userdist in model fitting

This is the key change to `secr.fit()`:

```
1 f0 <- secr.fit(  
2   full_ch,  
3   detectfn = "HHN",  
4   mask = mask_model,  
5   model = list(D ~ 1, lambda0 ~ 1, sigma ~ 1),  
6   details = list(userdist = userd_model),  
7   verify = FALSE,  
8   trace = FALSE  
9 )
```

The model still uses `mask_model`; only the movement distances have changed.

Extension 4: Multisession masks and distances

Extension 4: Multisession masks

Problem: `ch_ms` has multiple sessions, so model fitting needs one mask per session.

Implementation:

- build a trap-buffer polygon for each session
- crop the elevation-constrained mask to each session
- create one `userdist` matrix per session
- keep names aligned with `traps_ms`

One mask per session

Create polygons around traps active in each session:

```
mask_per_session_sf <- list()
for (i in seq_along(traps_ms)) {
  mask_per_session_sf[[i]] <- st_as_sf(traps_ms[[i]],
                                     coords = c("x", "y"), crs = my_crs) |>
    st_buffer(mask_buffer) |>
    st_union()
}
names(mask_per_session_sf) <- names(traps_ms)
```

Crop the full elevation mask:

```
mask_ms <- list()
for (i in seq_along(mask_per_session_sf)) {
  mask_in <- lengths(st_intersects(mask_elev_sf,
                                   mask_per_session_sf[[i]])) > 0
  mask_ms[[i]] <- subset(mask_elev, subset = mask_in)
}
names(mask_ms) <- names(traps_ms)
```

One userdist per session

Use the same mask indicator to subset the full distance matrix:

```
1 userd_ms <- list()
2 for (i in seq_along(mask_per_session_sf)) {
3   mask_in <- lengths(st_intersects(mask_elev_sf,
4                                     mask_per_session_sf[[i]])) > 0
5   userd_ms[[i]] <- userd_elev[, mask_in, drop = FALSE]
6 }
7 names(userd_ms) <- names(traps_ms)
```

The names of mask_ms, userd_ms, and traps_ms should all match.

Fit multisession models

The fitting call uses lists for mask and userdist:

```
1 ms0 <- secr.fit(  
2   ch_ms,  
3   detectfn = "HHN",  
4   mask = mask_ms,  
5   model = list(D ~ 1, lambda0 ~ 1, sigma ~ 1),  
6   details = list(userdist = userd_ms),  
7   verify = FALSE,  
8   trace = FALSE  
9 )
```

Multisession analyses also good when sessions are spatial (geographically separate surveys that you want to combine)

Multisession model set

Can share parameters across sessions:

```
ms1 <- secr.fit(ch_ms, detectfn = "HHN", mask = mask_ms,  
               model = list(D ~ std_tri, lambda0 ~ 1, sigma ~ 1),  
               details = list(userdist = userd_ms), start = ms0)
```

Or include session as a covariate:

```
ms5 <- secr.fit(ch_ms, detectfn = "HHN", mask = mask_ms,  
               model = list(D ~ session, lambda0 ~ 1, sigma ~ 1),  
               details = list(userdist = userd_ms), start = ms0)
```

Note: How to extrapolate from session-specific covariate models to a wider region is unclear.

Extension 5: Sex-specific models

Extension 5: Sex-specific models

Problem 1: detection or density may differ by sex **Problem 2:** sex not known for all animals

Two approaches:

1. `hcov = "sex"`: individual covariate models (Problem 1 & 2)
2. `groups = "sex"`: grouped real-parameter models (Problem 1 only)

Important: do not compare the `hcov` and `groups` models with each other (or with other `secr` models) using AIC.

Sex as an individual covariate

hcov = "sex" are finite mixture models that retain animals with unknown sex.

```
1 f0 <- secr.fit(  
2   full_ch,  
3   detectfn = "HHN",  
4   mask = mask_model,  
5   model = list(D ~ 1, lambda0 ~ h2, sigma ~ 1),  
6   hcov = "sex",  
7   details = list(userdist = userd_model),  
8   verify = FALSE  
9 )
```

h2 is the term used by secr for the first two levels of the individual covariate.

Sex as groups

Grouped models estimate separate real parameters by sex, but animals with unknown sex must be removed:

```
full_ch_known_sex <- subset(  
  full_ch,  
  subset = covariates(full_ch)$sex != "Unknown"  
)  
full_ch_known_sex <- shareFactorLevels(full_ch_known_sex)
```

Then fit grouped models:

```
1 f0g <- secr.fit(  
2   full_ch_known_sex,  
3   detectfn = "HHN",  
4   mask = mask_model,  
5   model = list(D ~ 1, lambda0 ~ g, sigma ~ 1),  
6   groups = "sex",  
7   details = list(userdist = userd_model)  
8 )
```

Sex-specific model comparison

Compare each sex-specific candidate set internally:

```
AIC(sex_f0, sex_f1, sex_f2, sex_f3, sex_f4, criterion = "AICc")
```

								model
sex_f3				D~1	lambda0~valley_stream + h2	sigma~1	pmix~h2	
sex_f4	D~std_tri			lambda0~cliff + valley_stream + h2	sigma~1	pmix~h2		
sex_f0				D~1	lambda0~h2	sigma~1	pmix~h2	
sex_f2				D~std_elev	lambda0~h2	sigma~1	pmix~h2	
sex_f1				D~std_tri	lambda0~h2	sigma~1	pmix~h2	
		detectfn	npar	logLik	AIC	AICc	dAICc	AICcwt
sex_f3	hazard	halfnormal	6	-122.3665	256.733	261.154	0.000	0.6539
sex_f4	hazard	halfnormal	8	-119.8368	255.674	264.144	2.990	0.1466
sex_f0	hazard	halfnormal	5	-125.6102	261.220	264.220	3.066	0.1412
sex_f2	hazard	halfnormal	6	-125.4404	262.881	267.302	6.148	0.0302
sex_f1	hazard	halfnormal	6	-125.5164	263.033	267.454	6.300	0.0280

Sex-specific model comparison

Compare each sex-specific candidate set internally:

```
AIC(sex_f0g, sex_f1g, sex_f2g, sex_f3g, sex_f4g, criterion = "AICc")
```

				model		detectfn
sex_f3g			D~1	lambda0~valley_stream + g	sigma~1	hazard halfnormal
sex_f4g	D~std_tri		lambda0~cliff + valley_stream + g	sigma~1	hazard	halfnormal
sex_f0g			D~1	lambda0~g	sigma~1	hazard halfnormal
sex_f1g			D~std_tri	lambda0~g	sigma~1	hazard halfnormal
sex_f2g			D~std_elev	lambda0~g	sigma~1	hazard halfnormal
	npars	logLik	AIC	AICc	dAICc	AICcwt
sex_f3g	5	-194.3884	398.777	402.306	0.000	0.8880
sex_f4g	7	-193.1579	400.316	407.782	5.476	0.0575
sex_f0g	4	-199.3268	406.654	408.876	6.570	0.0332
sex_f1g	5	-198.7941	407.588	411.118	8.812	0.0108
sex_f2g	5	-198.8246	407.649	411.179	8.873	0.0105

Sex-specific model comparison

Finite mixtures, best model:

```
predict(sex_f3)
```

```
$`session = 1, h2 = Female, valley_stream = 0`
```

	link	estimate	SE.estimate	lcl	ucl
D	log	1.592746e-04	4.377542e-05	9.385261e-05	2.703006e-04
lambda0	log	1.969504e-03	1.354382e-03	5.816625e-04	6.668724e-03
sigma	log	7.657208e+03	1.133651e+03	5.737628e+03	1.021900e+04
pmix	logit	3.761775e-01	1.285911e-01	1.708228e-01	6.383471e-01

```
$`session = 1, h2 = Male, valley_stream = 0`
```

	link	estimate	SE.estimate	lcl	ucl
D	log	1.592746e-04	4.377542e-05	9.385261e-05	2.703006e-04
lambda0	log	2.581066e-03	8.225013e-04	1.403074e-03	4.748076e-03
sigma	log	7.657208e+03	1.133651e+03	5.737628e+03	1.021900e+04
pmix	logit	6.238225e-01	1.285911e-01	3.616529e-01	8.291772e-01

Sex-specific model comparison

Groups, best model:

```
predict(sex_f3g)
```

```
$`session = 1, g = Female, valley_stream = 0`
```

	link	estimate	SE.estimate	lcl	ucl
D	log	8.181020e-05	2.537140e-05	4.517155e-05	1.481665e-04
lambda0	log	1.114858e-03	6.065400e-04	4.109557e-04	3.024436e-03
sigma	log	8.204458e+03	1.407858e+03	5.875501e+03	1.145658e+04

```
$`session = 1, g = Male, valley_stream = 0`
```

	link	estimate	SE.estimate	lcl	ucl
D	log	8.181020e-05	2.537140e-05	4.517155e-05	1.481665e-04
lambda0	log	2.253713e-03	7.589529e-04	1.185533e-03	4.284337e-03
sigma	log	8.204458e+03	1.407858e+03	5.875501e+03	1.145658e+04

Extension 6: Advanced reporting

Extension 6: Advanced reporting

Reporting follows the Session 2 pattern:

- choose fitted models
- choose reporting mask
- estimate abundance with `region.N()`
- convert abundance to density
- join with AICc
- make maps and covariate-effect plots

Main change: If reporting over a new mask, replace `userdist` with the matching distance matrix.

Other reporting code works more or less like before (see scripts and vignette for details).

Reporting mask and userdist

Choose the reporting mask:

```
extrap_mask <- mask_survey_area_elev  
extrap_mask_userd <- userd_survey_area_elev
```

Copy the fitted models and swap the distance matrix:

```
1 fitted_models_region <- lapply(fitted_models, function(model) {  
2   model$details$userdist <- extrap_mask_userd  
3   model  
4 })
```

Now fitted models stored in `fitted_models_region` can be used to extrapolate to the reporting mask (e.g. abundance, maps).

Comparing full, short, multisession surveys

Full survey:

	modelname	AICc	dAICc	estimate	D	CV
1	f3	225.08	0.00	33.32	0.02	25.91
2	f4	226.39	1.31	34.14	0.02	25.73
3	f0	228.76	3.68	33.23	0.02	25.78
4	f2	231.24	6.16	33.71	0.02	25.27
5	f1	231.38	6.30	33.55	0.02	25.29

Short survey:

	modelname	AICc	dAICc	estimate	D	CV
1	s3	138.29	0.00	45.19	0.02	36.39
2	s0	141.70	3.41	44.50	0.02	37.16
3	s4	141.78	3.48	48.67	0.02	38.67
4	s1	144.28	5.99	44.71	0.02	37.17
5	s2	144.79	6.49	44.44	0.02	36.13

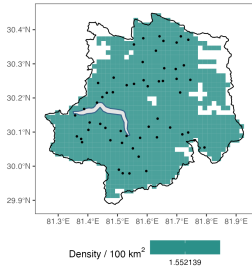
Multisession survey may return session-specific results (not here).

	modelname	AICc	dAICc	estimate	D	CV
1	ms0	221.72	4.20	40.64	0.02	32.88
2	ms1	224.25	6.74	40.69	0.02	32.20
3	ms2	224.25	6.74	40.76	0.02	32.33
4	ms3	217.82	0.31	41.07	0.02	32.78

Predicted density maps

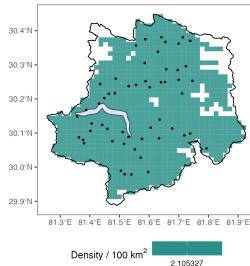
Full survey

Predicted density



Shortened survey

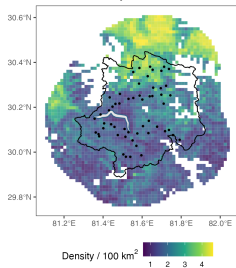
Predicted density



Predicted density maps

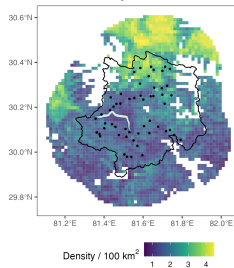
Multisession session 1

Predicted density from ms4 - session 1



Multisession session 2

Predicted density from ms4 - session 2



Workflow recap

1. Long surveys require thinking about closure.
2. Shortened surveys and multisession analyses change both captures and effort.
3. Habitat constraints change masks for fitting and reporting.
4. Barriers change movement distances, not the capture-history object.
5. Sex-specific models require careful handling of unknown sex and AIC comparisons.
6. Extrapolation to a new mask needs a `userdist` that matches.